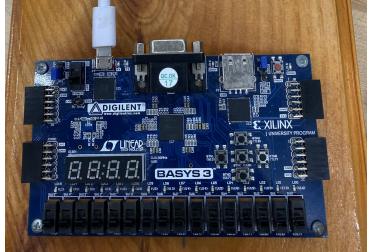


Objective: Introducing the Vivado software to write code to program it to the BASYS 3 circuit board to incorporate and test the logic gates: AND, NAND, OR, XOR, NOT.

Equipment: Vivado 2017-4 software used to program a BASYS 3 circuit board.



Procedure/Design (Part 1 AND): The outline for the program of the circuit board for the AND operation is to create 2 inputs and assign it to 1 output, to only turn on the light when both inputs/switches are turned on.

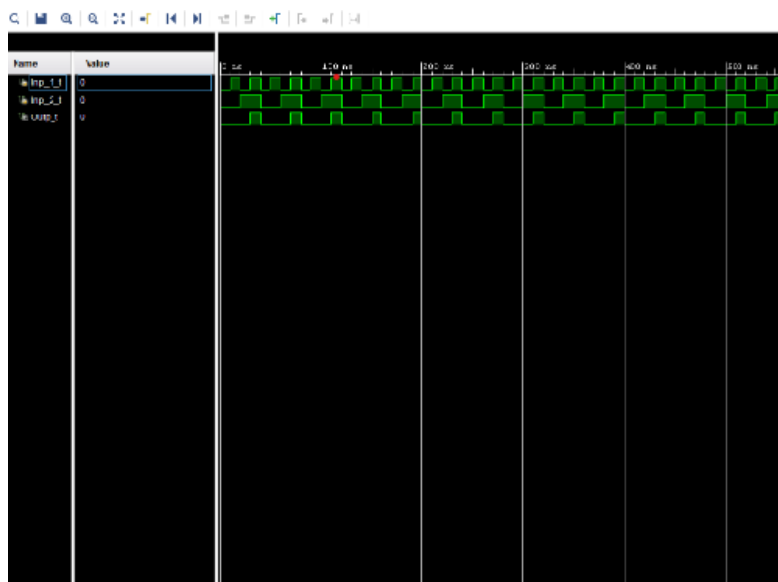
```
20 ///////////////////////////////////////////////////  
21  
22  
23 module Part_1(  
24     input Inp_1,  
25     input Inp_2,  
26     output Outp  
27 );  
28  
29     assign Outp = Inp_1 & Inp_2;  
30 endmodule  
31
```



```

2  module Part_1_Sim(
3  );
4  reg Inp_1_t;
5  reg Inp_2_t;
6  wire Outp_t;
7  Part_1 UUT(
8  .Inp_1(Inp_1_t),
9  .Inp_2(Inp_2_t),
10 .Outp(Outp_t)
11 );
12 initial
13 begin
14   Inp_1_t=1'b0;
15   Inp_2_t=1'b0;
16 end
17 always #5 Inp_1_t=~Inp_1_t;
18 always #10 Inp_2_t=~Inp_2_t;
19 endmodule
20
21

```



Results: The results of the AND operation is represented by the following truth table:

SW0	SW1	LED0
0	0	0
0	1	0
1	0	0
1	1	1

The output LED light only turns on when BOTH switches are turned on.

Design (Part 2 NAND): The next design is for the NAND operation. Essentially, this operation is the inverse of the AND operation. Hence, the output should only be off when the two input switches are turned on.

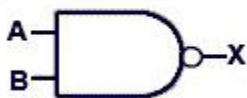
```

////////////////////////////////////////
`timescale 1ns / 1ps

module Part_2_1(
    input wire Inp_1,
    input wire Inp_2,
    output wire Outp
);
    assign Outp = ~((Inp_1 & Inp_2));
endmodule

module Part_2_Sim(
);
    reg Inp_1_t;
    reg Inp_2_t;
    wire Outp_t;
    Part_2_1 UUT(
        .Inp_1(Inp_1_t),
        .Inp_2(Inp_2_t),
        .Outp(Outp_t)
    );
    initial
    begin
        Inp_1_t=1'b0;
        Inp_2_t=1'b0;
    end
    always #5 Inp_1_t=~Inp_1_t;
    always #10 Inp_2_t=~Inp_2_t;
endmodule

```



Results: The results of the NAND operation is represented by the following truth table:

SW0	SW1	LED0
-----	-----	------

0	0	1
0	1	1
1	0	1
1	1	0

Design (Part 2 OR): The design for the OR operation will use 2 inputs and 1 output. When either input 1 or input 2 is on, then the output LED will turn on.

```

`timescale 1ns / 1ps

module Part_2_2(
    input wire Inp_1,
    input wire Inp_2,
    output wire Outp
);
    assign Outp = Inp_1 || Inp_2;
endmodule

module Part_1_Sim1
3   );
4   reg Inp_1_t;
5   reg Inp_2_t;
6   wire Outp_t;
7   Part_1 UUT(
8       .Inp_1(Inp_1_t),
9       .Inp_2(Inp_2_t),
10      .Outp(Outp_t)
11  );
12  initial
13  begin
14      Inp_1_t=1'b0;
15      Inp_2_t=1'b0;
16  end
17      always #5 Inp_1_t=~Inp_1_t;
18      always #10 Inp_2_t=~Inp_2_t;
19  endmodule
20  |
21

```



Results: The results of the OR operation is represented by the following truth table:

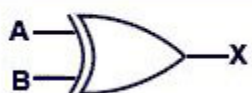
SW0	SW1	LED0
0	0	0
1	0	1
0	1	1
1	1	1

Design (Part 2 XOR): The design for the XOR operation uses two inputs and 1 output. The LED output turns on when ONE of the two input switches are turned on. When both switches are on the output is off.

```

11 `timescale 1ns / 1ps
12
13 module Part_2_3(
14     input wire Inp_1,
15     input wire Inp_2,
16     output wire Outp
17 );
18     assign Outp = Inp_1 ^ Inp_2;
19 endmodule
20
21 module Part_1_Sim(
22 );
23     reg Inp_1_t;
24     reg Inp_2_t;
25     wire Outp_t;
26     Part_1 UUT(
27         .Inp_1(Inp_1_t),
28         .Inp_2(Inp_2_t),
29         .Outp(Outp_t)
30     );
31     initial
32     begin
33         Inp_1_t=1'b0;
34         Inp_2_t=1'b0;
35     end
36     always #5 Inp_1_t=~Inp_1_t;
37     always #10 Inp_2_t=~Inp_2_t;
38 endmodule
39
40
41

```



Results: The results of the XOR operation is represented by the following truth table:

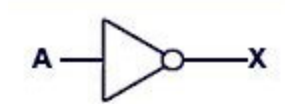
SW0	SW1	LED0
0	0	0
1	0	1
0	1	1
1	1	0

Design (Part 2 NOT): The design for this program is using one input and one output. When the input is on then the LED output should be off, and when the input is off the LED output should be on. The output is the INVERSE of the input.

```

21  `timescale 1ns / 1ps
22  |
23  module Part_2_4(
24      input wire Inp_1,
25      output wire Outp
26  );
27      assign Outp = ~Inp_1;
28  endmodule
29
30  module Part_1_Sim(
31  );
32      reg Inp_1_t;
33      reg Inp_2_t;
34      wire Outp_t;
35      Part_1 UUT(
36          .Inp_1(Inp_1_t),
37          .Inp_2(Inp_2_t),
38          .Outp(Outp_t)
39      );
40      initial
41      begin
42          Inp_1_t=1'b0;
43          Inp_2_t=1'b0;
44      end
45      always #5 Inp_1_t=~Inp_1_t;
46      always #10 Inp_2_t=~Inp_2_t;
47  endmodule
48  |
49

```



Results: The results of the NOT operation is represented by the following truth table:

SW0	LED0
0	1
1	0

Design (Part 3 (2 Input - 5 Output)): The design for this part uses 2 initialized inputs and 5 different outputs. The 5 outputs correspond to each of the discussed logic gate operations that went over in the previous parts of this lab. Using the code from our previous designs, we assign the operation to an individual output.

```

1  `timescale 1ns / 1ps
2
3
4  module Part3Lab1(
5      input wire Inp_1,
6      input wire Inp_2,
7      output wire Outp0,
8          output wire Outp1,
9          output wire Outp2,
10         output wire Outp3,
11         output wire Outp4
12  );
13      assign Outp0 = Inp_1 & Inp_2;
14      assign Outp1 = ~((Inp_1 & Inp_2));
15      assign Outp2 = Inp_1 | Inp_2;
16      assign Outp3 = Inp_1 ^ Inp_2;
17      assign Outp4 = ~Inp_1;
18  endmodule

```

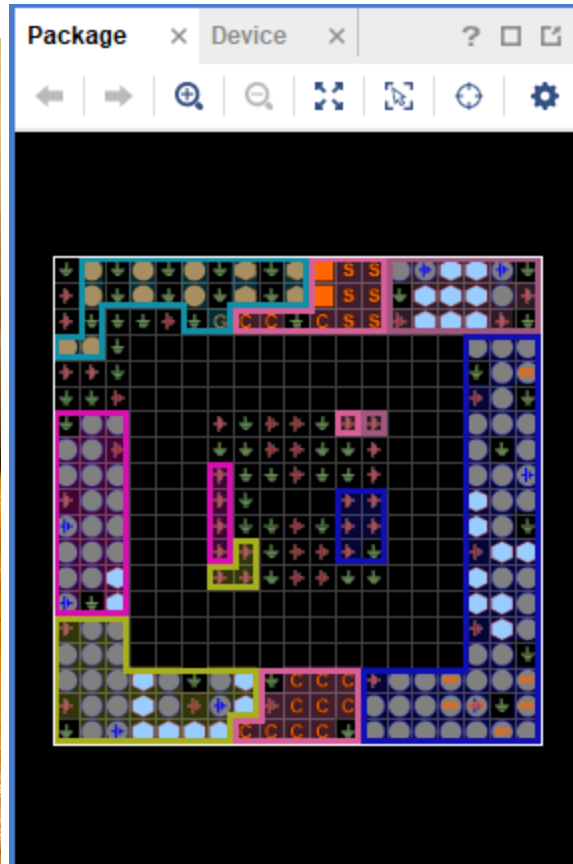
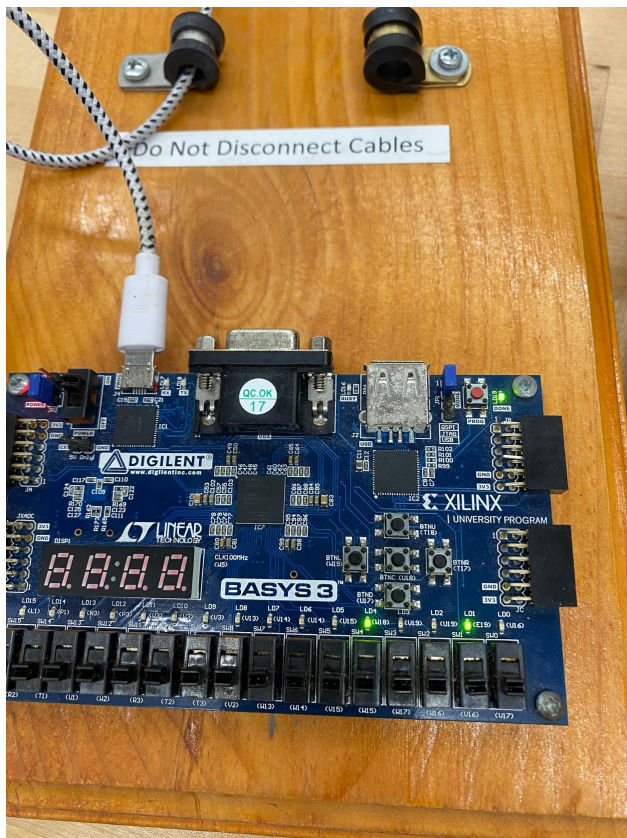
Lab1_Sim.v

C:/Users/ry266542/Desktop/LAB/lab1/lab1.srscs/sim_1/new/Lab1_Sim.v

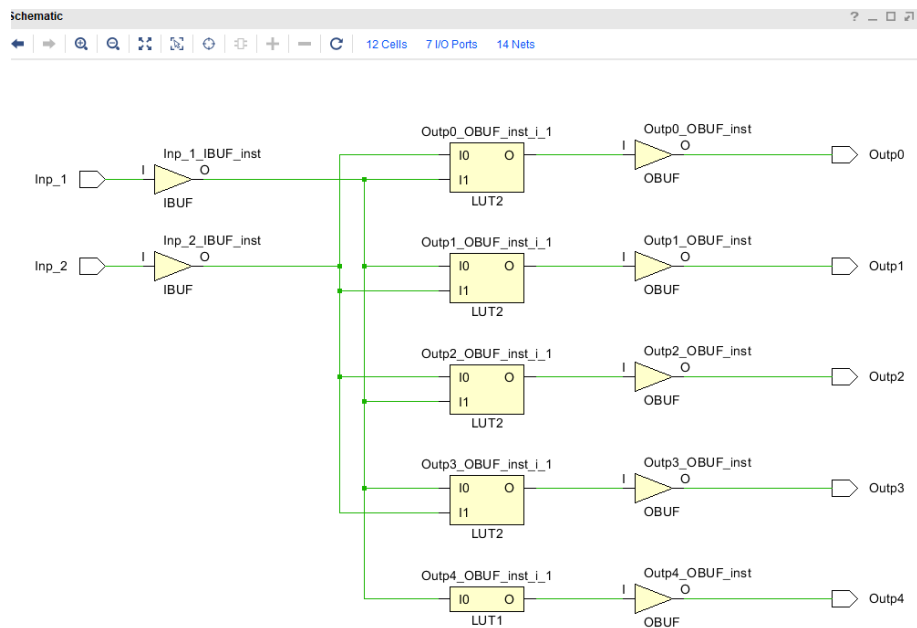


```
1  `timescale 1ns / 1ps
2  module Lab1_Sim(
3  );
4  reg Inp_1_t;
5  reg Inp_2_t;
6
7  wire Outp0_t;
8  wire Outp1_t;
9  wire Outp2_t;
10 wire Outp3_t;
11 wire Outp4_t;
12
13 Lab1 UUT(
14 .Inp_1(Inp_1_t),
15 .Inp_2(Inp_2_t),
16 .Outp0(Outp0_t),
17 .Outp1(Outp1_t),
18 .Outp2(Outp2_t),
19 .Outp3(Outp3_t),
20 .Outp4(Outp4_t)
21 );
22 );
23 initial
24 begin
25 Inp_1_t=1'b0;
26 Inp_2_t=1'b0;
27 end
28 always #5 Inp_1_t=~Inp_1_t;
29 always #10 Inp_2_t=~Inp_2_t;
30 endmodule
```

☒	Inp_1	IN		V17	▼	☒	14	LVC MOS33*
☒	Inp_2	IN		W16	▼	☒	14	LVC MOS33*
☒	Outp0	OUT		U16	▼	☒	14	LVC MOS33*
☒	Outp1	OUT		E19	▼	☒	14	LVC MOS33*
☒	Outp2	OUT		U19	▼	☒	14	LVC MOS33*
☒	Outp3	OUT		V19	▼	☒	14	LVC MOS33*
☒	Outp4	OUT		W18	▼	☒	14	LVC MOS33*



Both switches are off, Only NOT operator and NAND are on.



Results: The results for all of the previous operations in 1 program is represented by the following truth table:

A	B	LED0&	LED1~&	LED2	LED3^	LED4~
0	0	0	1	0	0	1
1	0	0	1	1	1	0
0	1	0	1	1	1	
1	1	1	0	1	0	

Conclusion: The experiment was successful. I was able to design and test the logic gates on the BASYS 3 Circuit board and check if the inputs gave the desired output where all the results were accurate based on the rules of each logic gate.